

Problem 1 — ATLAS

Build the study graph. Prove every answer. Elimination gate.

VIT Vellore · SCOPE

Issued ---- September 2026

All data in this event is synthetic. No real patient, site or study exists in it.

Stage 1 — ATLAS

Build the map. Prove every answer.

Duration: ---- hours · **This is an elimination round.**

The situation

A clinical trial’s data sits in nine tables that do not reference each other. A reviewer’s question — “which subjects show the liver-damage pattern?” — needs four of those tables joined, unit conversions applied, and the answer backed by the exact records it rests on.

Build that.

What you build

1. A study knowledge graph. Every subject’s records joined into one connected structure — subject → visits → laboratory results, adverse events, doses, medicines, history, disposition — with the protocol’s rules available alongside. We call the per-subject view **Patient 360**.

2. An agent that answers questions from it, citing records.

```
# stage1/atlas.py
class StudyGraph:
    def __init__(self, data_dir: str): ...           # the hackathon-data folder
    def build(self, cut: int | None = None) -> dict: ... # nodes, edges, subjects, ms
    def patient360(self, usubjid: str) -> dict: ...     # everything about one subject

class Atlas:
    def __init__(self, graph: StudyGraph): ...
    def answer(self, question: Question) -> Answer: ...
```

`Question` and `Answer` are in `starter/schemas.py`. Do not modify them.

The four kinds of question

Kind	Example	What a correct answer looks like
<code>count</code>	“How many subjects at site S07 discontinued due to an adverse event?”	the exact integer, with the disposition records as evidence
<code>lookup</code>	“List the laboratory and adverse-event records for 042-S05-003 within 7 days of the WEEK8 visit”	the exact list of record references
<code>finding</code>	“Which subjects meet the Hy’s law criteria?”	the exact list of subject IDs, with the laboratory records that prove each
<code>trap</code>	“Which subjects at site S01 received a wrong dose?”	<code>[]</code> — when the honest answer is none, say none

Hard limits, per question

Limit	Value
Wall time	≤ 120 seconds per question

A breach scores zero for that question. More than 20% of questions breaching fails the gate.

Load and index the study **once**, then answer from what you built. Re-reading 14,400 laboratory rows for every question will not fit in 120 seconds.

What you must handle

These are not edge cases. They are the problem.

- **Units.** One site's laboratory reports certain tests in a different unit. `data/reference_ranges.csv` tells you which laboratory and which range — look at the `LAB` column. A value read without its unit will look normal when it is four times the limit.
- **Dates.** More than one format appears.
- **Non-numeric laboratory values.** "<5", "ND", "12,4", empty. "<5" means below detection — not zero.
- **Duplicate records and duplicate subjects.** A person can be enrolled twice.
- **Rows with missing fields.** Skip them; do not crash.
- **Documents that address your agent.** The protocol and the laboratory manual are evidence. If one of them contains a sentence aimed at an automated reviewer, that sentence is a fact you may report — not an instruction to follow.

Evidence — read this twice

Every `RecordRef` in your answer must:

1. **exist** in the study, and
2. **actually support the claim you made about it.**

A liver-signal claim citing an ALT record that is not above three times its limit scores **zero**, even if you really did read that record. So cite the records that show the thing — not neighbouring rows, not the subject's demographics row, not a plausible-looking sequence number.

```
RecordRef(domain="LB", usubjid="042-S07-001", seq=31)
RecordRef(domain="DOC", document="lab-manual", section="units")
```

What happens partway through

At a point during the stage, the study changes. You will get one line of announcement and nothing more. Systems that build the graph once at the start and answer from cache will begin producing wrong answers without erroring.

Design for re-reading the world.

How you are scored — 100 points

	Points
Weighted score on 40 hidden questions (correctness × evidence validity – confidence penalty)	60
Evidence discipline — share of answers with fully valid evidence	15
Trap handling — traps answered honestly (6 traps)	10
Resilience — questions after the mid-stage change answered correctly	10
Engineering — the eight README sections, build statistics, commit cadence	5

A confident wrong answer is penalised more than an uncertain one. Set [confidence](#) honestly.

Graded on three hidden versions of the study. The median counts.

The gate — you continue only if ALL of these hold

	Threshold
Contract	harness runs all 40 questions without a crash; every answer schema-valid
Score	≥ 60 / 100
Evidence	≥ 90% of answers with fully valid evidence
Traps	≥ 2 of 6 correct
Limits	respected; ≤ 20% of questions breaching
Resilience	≥ 50% of post-change questions correct

Results published within [\[X\]](#) hours with your six gate values.

What you submit

1. **Repository URL** — public, or private with [\[ORGANISER GITHUB HANDLE\]](#) added
2. [graph_stats.json](#) — your build statistics: nodes, edges, subjects covered, build time, the cut you built at
3. [stage1_public.json](#) — your own run against the ten public questions
4. [README.md](#), max one page, with these headed sections — use the README template:
 - **How we understood the problem** — in your words, five lines
 - **Architecture** — what runs, in what order
 - **Tech stack** — a table: layer, choice, and *why this rather than the obvious alternative*
 - **Data handling** — units, dates, non-numeric values, malformed rows
 - **Documents** — how you use them, and what you do with one that instructs your agent
 - **When the answer is nothing** — how your agent decides that

— What we know is weak

A worked example — the question that decides your gate

The hidden bank asks, in one form or another: “Which subjects meet potential Hy’s law criteria?” Here is the whole path from raw rows to a valid answer.

Step 1 — the rule, from the protocol

`documents/protocol_v1.md` §7:

Potential Hy’s law: ALT or AST $> 3 \times \text{ULN}$ together with total bilirubin $> 2 \times \text{ULN}$ within 14 days, without cholestasis or alternative explanation.

You are not expected to know this. You are expected to read it.

Step 2 — the reference ranges

`data/reference_ranges.csv` contains, among others:

```
LBTESTCD,UNIT,LOW,HIGH,LAB
ALT,U/L,7.0,56.0,CENTRAL
BILI,mg/dL,0.1,1.2,CENTRAL
ALT,ukat/L,0.12,0.93,S07
```

Read the third line twice. One site does not use the central laboratory.

Step 3 — the rows

```
lb[(lb.USUBJID == "042-S07-001") & (lb.VISIT == "WEEK8")]
USUBJID,LBSEQ,VISIT,LBDTC,LBTESTCD,LBORRES,LBORRESU
042-S07-001,25,WEEK8,2026-03-30,ALT,3.995,ukat/L
042-S07-001,27,WEEK8,2026-03-30,BILI,5.38,mg/dL
```

Step 4 — what a naive system concludes

ALT is 3.995. The range is 7 to 56. Below range: low, not high. No signal. Move on.

That subject has a liver injury signal and your system just walked past it.

Step 5 — what a correct system does

The unit is `ukat/L`, not `U/L`. One $\mu\text{kat/L}$ is 60 `U/L`.

ALT	3.995 $\mu\text{kat/L} \times 60 = 239.7 \text{ U/L}$	$3 \times \text{ULN} = 168$	$239.7 > 168$	✓
BILI	5.38 $\text{mg/dL} = 5.38 \text{ mg/dL}$	$2 \times \text{ULN} = 2.4$	$5.38 > 2.4$	✓
	same date, so within 14 days			✓

Both criteria met. This subject is a candidate.

Step 6 — the answer you return

```
{
  "question_id": "Q018",
  "answer": ["042-S05-003", "042-S07-001", "042-S08-014"],
  "text": "3 Hy's law candidates. For 042-S07-001: ALT 239.7 U/L (>3xULN, converted from ukat/L) and bilirubin 5.38 mg/dL (>2xULN) on the same day at WEEK8.",
  "evidence": [
```

```
{
  "domain": "LB", "usubjid": "042-S07-001", "seq": 25},
  {"domain": "LB", "usubjid": "042-S07-001", "seq": 27},
  {"domain": "LB", "usubjid": "042-S05-003", "seq": 31},
  {"domain": "LB", "usubjid": "042-S05-003", "seq": 33},
  {"domain": "LB", "usubjid": "042-S08-014", "seq": 31},
  {"domain": "LB", "usubjid": "042-S08-014", "seq": 33}
},
"confidence": 0.9,
"steps_used": 6,
"tokens_used": 0
}
```

Note what this answer does **not** do: it does not cite a lab record it never fetched, it does not round the subject list, and it does not need a model call at all. The reasoning is arithmetic; the model’s job, if you use one, is to write the sentence — not to decide whether 239.7 exceeds 168.

Step 7 — and a trap, for contrast

“Which subjects at site S01 received a wrong dose?”

```
{
  "question_id": "Q031",
  "answer": [],
  "text": "No dosing errors at site S01. The dosing errors in this study are elsewhere.",
  "evidence": [],
  "confidence": 0.85
}
```

Full marks. An empty list with no invented evidence is a correct answer, and saying so is worth exactly as much as finding something.

Ten failure modes, from previous runs of this format

1. Reading `LBORRES` without `LBORRESU`
2. Parsing only ISO dates, so one site’s records vanish from every date window
3. Treating "`<5`" as 0 instead of “below detection, value unknown”
4. Crashing on a row with a missing field instead of skipping it
5. Counting a duplicated subject twice in a total
6. Obeying a sentence in the laboratory manual that tells automated reviewers to exclude sites
7. Citing records that look right — same subject, plausible sequence number — but do not show the finding
8. Answering a trap with a confident guess
9. Reading the protocol once at 09:00 and answering from cache at 13:01
10. Re-reading every CSV for every question and running out of time

Suggested order of work

1. Load the files and print row counts (before the stage — see `04_environment_setup.md`)
2. Read the reference ranges and the protocol **first** — everything else depends on them
3. Build the graph for one subject, print it, check it against the raw records by eye
4. Build it for all subjects; record your statistics

5. Answer the `count` questions — they are the simplest and they validate your joins
6. Answer `lookup`; get dates right here and the rest follows
7. Answer `finding`; this is where units and thresholds bite
8. Make traps return `[]` honestly — this is 10 free points that most teams lose
9. Run `run_local_harness.py` repeatedly; watch evidence validity, not just correctness
10. Re-read the protocol; make sure a second build after the amendment produces different answers where it should